
Presentation Script

(Slide 1: Title Slide)

"Hi everyone, good morning/afternoon. My name is [Your Name], and for my [Class Name] mini-project, I developed an ATM Simulation using C programming. Today, I'm excited to walk you through its design, logic, and how it works."

(Slide 2: Agenda)

"To see how the project came together, here's the agenda for today. I'll start with the main goals of the project and the features I implemented.

Then, we'll dive into the most important part: the program's logic, looking at a flowchart and the specific C concepts I was required to use. I'll highlight some key pieces of code and then give a live demonstration. Finally, I'll touch on a few challenges I faced and how this project could be expanded in the future, before opening it up for any questions."

(Slide 3: Project Objective)

"The main objective of this project was to apply our fundamental programming concepts to a practical, real-world scenario.

The goal wasn't just to *make* an ATM, but to build it under a very specific technical constraint: the entire program's flow had to be controlled *only* using if-else statements, while loops, and do-while loops. This meant no for loops or switch-case statements, forcing a focus on these core building blocks."

(Slide 4: Features Implemented)

"So, what does the simulation do?

It has a secure PIN verification system that gives the user three attempts before locking them out. Once in, it supports all the essential transactions:

- **Cash Withdrawal**, which includes a check for sufficient funds.
- **Cash Deposit**, which updates the balance.
- A simple **Balance Inquiry** to see the current total.

- And a **Fund Transfer** to simulate moving money to another account.

Finally, it uses a session management loop, so a user can perform multiple transactions in one sitting without having to log in again."

(Slide 5: Program Flow (Flowchart))

"This flowchart shows the entire logic of the program from start to finish. It's really broken into two main parts.

The first part, on the left, is the **PIN Entry**. This is controlled by a **do-while loop**. I chose a do-while loop because it's perfect for this job—it guarantees the loop runs at least once, so the user is always prompted for their PIN. If the PIN is wrong, the loop repeats until they get it right, or until they fail three times.

If the PIN is correct, we enter the second part of the program: the **Main Transaction Menu**, which is controlled by a single **while loop**. This loop will run continuously, showing the menu and letting the user select transactions... until they finally choose the 'Exit' option. At that point, the loop's condition becomes false, and the program ends."

(Slide 6: Core C Concepts Applied)

"So, to summarize how those requirements were met:

- The **do-while loop** was used for the PIN entry system. Again, this was perfect because it must execute at least once.
- The **while loop** was the 'engine' for the main transaction session. The entire session runs inside while (choice != 5).
- And finally, **if-else and else-if statements** were the 'brains' of the operation. They were used everywhere:
 1. To check if the PIN was correct.
 2. To 'route' the user to the correct transaction based on their menu choice.
 3. And for all validation, like the crucial check: if (withdrawalAmount <= balance)."

(Slide 7: Challenges & Solutions)

"Of course, the project had a few challenges. The biggest one by far was handling bad user input.

- **The Challenge:** If the program expected a number, but the user typed a letter like 'a', the scanf function would fail, and the program would get stuck in an infinite loop.
- **The Solution:** I had to add code to check the return value of scanf. If it failed, I had to write a small utility to 'clear the input buffer', which basically scrubs the bad input and

allows the loop to ask the user for a number again, safely."

(Slide 9: Thank You)

"That concludes my presentation. Thank you so much for your time and attention. I'd be happy to answer any questions."